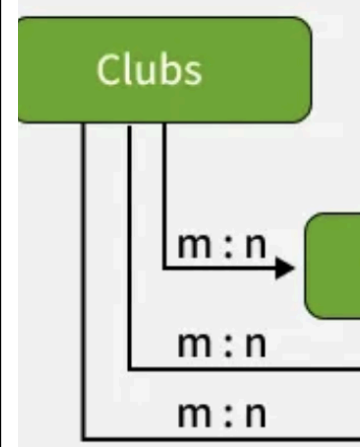
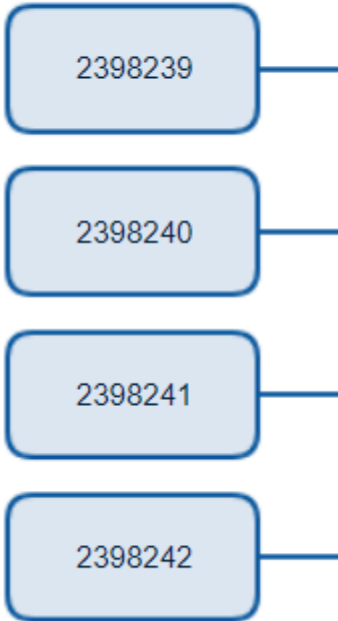
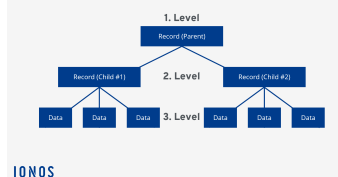


Distributed DB	• The data is stored across multiple nodes • Data are replicated and sharded • It combines multiple dbms	• It works correctly and continuously in time • We can add nodes easily • High performances • Handles large amount of data	• Duplicated data • Complexity to manage and configurate respect to a centralised dbms • Maintain the security in all nodes	 Finances The picture above represents the distributed database's structure i
----------------	--	---	---	---

Network DB	• Data is stored as a graph of entities and relationships • Improvement of the hierarchical model	• Allows records to have multiple parent and child links • Supports many-to-many relationships • Fast data access (pointer-based navigation) • Flexible (if not populated)	• Complex implementation (due to pointers) • Difficult to manage if maintenance is needed • Not flexible if too populated	 The diagram illustrates a network database structure. A green rounded rectangle labeled 'Clubs' is at the top. Three lines descend from it, each ending in an arrow pointing to a smaller green rounded rectangle. Each arrow is labeled 'm:n', representing many-to-many relationships between the 'Clubs' entity and the other three entities.
Key-value DB	• Data are stored as a collection of pairs of keys and values • The key is a unique identifier. It's often built by combining a category and an ID • The value is the content of the key • Value can be: string, number, list, array, JSON, binary file	• Simplicity: no predefined schema • High performance: optimised for fast operations • Scalability: different formats of data • Flexibility: works well in distributed system, across many servers	• Limited aggregation schema: it isn't possible to connect data together because of a lack of schema • Difficulty in Backup: it's easy to use and optimised for fast operations, so the lack of specific features can complicate backup and recovery	• caching • Session/local storage • user profile "user:101" → {name: "Anna", age: 20} user:101 is the key {name: "Anna", age: 20} is the value

				Keys 
Hierarchical databases	• The data (records) is stored in a tree-like structure • Parent-child relationships: 1 to many	• Easy to understand • Fast data retrieval • Higher data integrity • Easy implementation of security policies base on hierarchy	• Lack of flexibility • Difficult to manage • Possible complex implementation • Query limitations • Difficult to represent many to many relationships	 IONOS The structure above shows how a Hierarchical database is represented and the relationship between parents and childs
Self Driving DB	• Cloud Databases that manage themselves • Use Ai & automatition	• Reduce human errors • Save time	• Expensive at the beginning • Issue of skills: Administrators and	• Oracle Autonomus Database • Amazon Aurora • Microsoft Azura SQL Database

		• Improve performance • Improve Security • Reduces costs in long term	IT staff need to learn new skills to work with these systems • Trust. Many companies are not comfortable letting a system make important decisions automatically.	
Cloud Database	• Can be noSQL or SQL • Can be a mix between different type of database • The security has to be guaranteed by the provider • There are different type of cloud database ○ Self managed ○ automated ○ Managed ○ Autonomous	• Accessible from anywhere • Easy maintenance: automatic updates and backups • Good scalability: you can easily increase the resources in the database	• Internet dependency: you need internet to access the data • Latency: if the servers are far the access to the data can be slower	some examples: • Amazon RDS • Google Cloud SQL • Oracle Autonomous Database

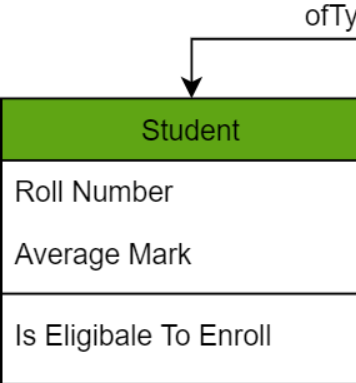
Document database	• Data is stored in documents • Documents are grouped into collections • Each document can have a different structure • Data is usually stored in JSON, BSON, or XML format	• Flexible schema, easy to modify and scale • Fast development, structure matches programming objects • Good performance with large and complex datasets • Horizontal scalability through sharding and replication • Supports nested data and complex queries on document fields	• No strong relationships like joins between documents • Data duplication may occur • Transactions across multiple documents can be limited • Consistency can vary in distributed systems	Instance (primary) Example systems: • MongoDB • CouchDB • Amazon DocumentDB
-------------------	--	--	--	--

				Document 1 <pre>{ "id": "1", "name": "John Sm "isActive": true, "dob": "1964-30-0 }</pre>
--	--	--	--	--

spatial DBMS	• Supports spatial data types (point, line, polygon) • Uses spatial indexes • Allows spatial queries (distance, overlap, containment) • Integrates with GIS (Geographic Information System) tools	• Efficient storage and querying of spatial data • Enables spatial analysis and mapping • Improves performance with spatial indexes	• More complex to manage • Requires more storage and processing power • Often higher cost or limited standardization	POINTS LINES POLYGONS
--------------	--	---	--	--

				HYDROLOG Rivers TRANSPORTAT Main Roads & Ra
--	--	--	--	---

Graph DBs	• NoSQL database • Query Language used: Cypher • Data saved as: • Nodes (things, for example Person, Product etc..) • Edges (connections for example BUYS, LIVES_IN) • Properties (details of nodes and edges, for example name, age)	• Fast to move between connections • Flexible, no fixed schema • Works best when data is highly connected • Faster queries with the relationships	• not good for analytics and big maths (sum of million of rows) • Different thinking and approach than SQL	
-----------	--	--	---	---

Object Oriented DB	• Data represented as objects • Relations as pointers • Each object has attributes and methods • OOP principles apply	• Natural representation of complex data • Reusability of objects • Easy to modify the relation between objects thanks to the use of pointers • Easier to represent multiple relationship • Better performance with big databases / multiple nested objects • Easier to program using OOP languages.	• Less intuitive for users • Lack of proper standardisation • Worse performance with simple queries • Limited diffusion/expertise	Each entity is represented as a class, such as Person, with multiple attributes like Name, Phone Number and Email Address. Each person instance is represented as an object of that class. It stores a pointer to objects representing the Address to represent its connections.  <pre> classDiagram class Student { Roll Number Average Mark Is Eligible To Enroll } </pre> The diagram shows a class named 'Student' with a green header. It has three attributes: 'Roll Number', 'Average Mark', and 'Is Eligible To Enroll'. An arrow labeled 'ofTy' points to the class name.
--------------------	--	---	--	--

Time-Series Database	- Relational database. - Database designed to store and manage time-stamped or time-indexed data efficiently. - Data is usually stored as sequences of values associated with timestamps.	- Ideal for analysing changes over time. - Handles large volumes of sequential data efficiently and compression of it. - Allows fast aggregation, downsampling and querying over time ranges. - Good for monitoring, IoT, financial data, and performance metrics.	- Less flexible for non-time-based data. - Can require specific query languages or tools. - May have limited relational features compared to traditional databases. - Storage can grow quickly with high-frequency data.	Time Series Db Ex Table 1: Sensors <table><thead><tr><th><u>sensor_id</u></th></tr></thead><tbody><tr><td>S01</td></tr><tr><td>S02</td></tr><tr><td>S03</td></tr></tbody></table> Table 2: SensorD <table><thead><tr><th><u>data_id</u></th><th>se</th></tr></thead><tbody><tr><td>1</td><td>S</td></tr><tr><td>2</td><td>S</td></tr><tr><td>3</td><td>S</td></tr></tbody></table>	<u>sensor_id</u>	S01	S02	S03	<u>data_id</u>	se	1	S	2	S	3	S
<u>sensor_id</u>																
S01																
S02																
S03																
<u>data_id</u>	se															
1	S															
2	S															
3	S															
Wide Column Stores	Rows can have different columns	- Great Scalability - Optimised to handle big data	Complex development	DBMS examples 1) apache Cassandra 2) Google big data												

	• Data is stored in column families • NoSQL	• Flexible schema • Easy to expand	• Don't support complex queries • High storage requirement	3) Apache HBase
Open Source Database	• The source code is free • Maintained by developers or founders • Support for relational or NoSQL • Cross-platform compatibility	• Free or low cost • Customisation thank to open code • Easy to inspect for security • Strong community support	• Public code visibility • Lack of official support	Relational: • MySQL (used in websites) • PostgreSQL (used in finance and research) • MariaDB NoSQL • MongoDB (used in web application) • CouchDB (used for distributed systems) • Cassandra (used by companies like Netflix)

DBMS: DataBase Management System

Software specialised in the management of a Database
Necessary to create, manage, access the data structures.

GLOSSARY

Cross reference

Riferimenti incrociati

Storage	Archiviazione (anche spazio di memoria)
Leaf	Foglia
Branch	Ramo
Crossing	Attraversamento